INDIAN INSTITUTE OF INFORMATION TECHNOLOGY KOTTAYAM

(An Institute of National Importance)

IOE-414: DEEP LEARNING

MINOR PROJECT REPORT



Project Title:
PatchTST Replication for Long Term Forecasting

Group Members:
1. Vibhaas Nirantar Srivastava (2023BCS0037)
2. Jayanth P (2023BCS0040)
3. Vinayak Sharma (2023BCS0002)
4. Vishwanath Darur (2023BCS0057)

Department of Computer Science and Engineering

October–2026

Table of Contents

Abstract 3

Base Paper Details 3

Problem Statement 3

Method Overview with Architecture 3

Dataset Details 4

Results and Visualizations 4

Implementation and Dataset Links 5

References 5

Abstract

We checked the PatchTST paper results using the authors' code on one RTX 5050 laptop GPU. The model reads a time series as small patches, and the same model is used for each variable. We ran the ETT datasets, Weather, Electricity, ablation tests, and a short masked pre-training test. The ETT PatchTST scores were within 3% MSE of the paper. On ETTh1 with a 96-step forecast, we got 0.376 MSE while the paper has 0.375. Removing patching and channel independence made the result worse. The results are close even though our training was much shorter.

Base Paper Details

- **Base Paper Title:** A Time Series is Worth 64 Words: Long-term Forecasting with Transformers
- **DOI link of the paper:** https://doi.org/10.48550/arXiv.2211.14730
- **Conference/Journal Name:** International Conference on Learning Representations (ICLR)
- **Authors:** Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, Jayant Kalagnanam
- **Year of Publication:** 2023

Problem Statement

Long-term forecasting means using past values to predict many future values. A normal Transformer can treat each time step as one token, but this becomes costly when we give it a long history. There are also many variables in datasets like weather and electricity. The paper asks if a Transformer can work better by reading short groups of values and by sharing one model across the variables. Our project checks these main claims with a small GPU and limited training time.

Method Overview with Architecture

PatchTST cuts each variable into patches of 16 values with stride 8. Each patch becomes one input token. A shared Transformer then learns from these tokens, and a prediction head gives the future values. "Channel independence" means the same weights are used for every variable, though each variable is processed separately. This keeps the attention smaller and lets the model look at more past data.

Past values -> 16-step patches -> shared Transformer -> future values

We used a 336-step look-back for the main runs and forecast 96, 192, 336 or 720 steps. We also tested a masked patch pre-training run, where 40% of the patches are hidden and the model learns to fill them. Our code comes from the paper authors, with small fixes for current libraries. Training epochs and some training windows were reduced to fit the laptop.

Dataset Details

ETTh1, ETTh2, ETTm1 and ETTm2 are transformer temperature datasets with seven variables each. ETTh is hourly and ETTm is recorded every 15 minutes. Weather has 21 variables and Electricity has 321. We used the standard train, validation and test split from the authors' code. For PatchTST on ETTm and Weather we used every second training window, but kept the full validation and test sets. One Electricity test trained on only 32 channels and was still tested on all 321. A later run trained on all channels.

Results and Visualizations

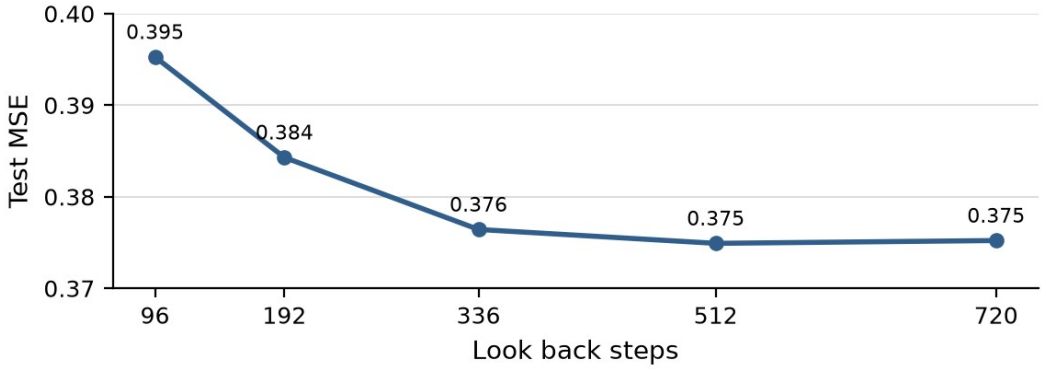
Table 1 shows the 96-step test MSE. Smaller MSE means the forecast is closer to the real data. PatchTST/42 beat our DLinear run on all four ETT sets and on Weather. The Paper column gives the paper's PatchTST/42 result. The Electricity 32-channel row is our own test, so there is no paper number for that setup.

Table 1. Test MSE for a 96-step forecast

Dataset	PatchTST ours	Paper	DLinear ours
ETTh1	0.376	0.375	0.438
ETTh2	0.276	0.274	0.300
ETTm1	0.292	0.290	0.300
ETTm2	0.163	0.165	0.173
Weather	0.155	0.152	0.175
Electricity 32 ch	0.145	—	0.142
Electricity full	0.137	0.130	0.142

In ETTh1 at horizon 96, the full model got 0.376 MSE. Without patching it got 0.387, without channel independence it got 0.416, and without both it got 0.484. This supports the two design choices. The look-back graph below shows that a longer history helps, mainly until about 336 steps.

Figure 1. ETTh1 96-step forecast MSE for different look-back lengths



The 32-channel Electricity model got 0.145 MSE when tested on all 321 channels. Training on all channels gave 0.137. These runs also had different window stride and training time, so this is not only a channel-count comparison. The short self-supervised test gave 0.368 MSE for a linear probe on ETTh1/96, close to the paper's 0.371.

Implementation and Dataset Links [GitHub]

Our implementation and result CSVs: https://github.com/vibhaas/patchtst-replication. Authors' code: https://github.com/yuqinie98/PatchTST. Dataset mirror: https://huggingface.co/datasets/thuml/Time-Series-Library. The comparison file in our repository lists every run beside the paper number. We did not run Traffic because the time budget ended. The biggest limit is shorter training than the paper, especially for full Electricity.

References [APA Style]

Nie, Y., Nguyen, N. H., Sinthong, P., & Kalagnanam, J. (2023). A time series is worth 64 words: Long-term forecasting with transformers. International Conference on Learning Representations. https://doi.org/10.48550/arXiv.2211.14730

Nie, Y. (2023). PatchTST [Source code]. GitHub. https://github.com/yuqinie98/PatchTST

THUML. (n.d.). Time-Series-Library [Data set]. Hugging Face. https://huggingface.co/datasets/thuml/Time-Series-Library